基于 TMDB 数据集的电影数据分析

https://dblab.xmu.edu.cn/blog/2738/

一、环境搭建

假设目前的环境是裸机。

（1）安装Linux操作系统：比如可以安装Ubuntu 16.04，可以参考这篇博客，完成操作系统的安装。
（2）安装Hadoop：需要在Linux系统上安装Hadoop，可以参考这篇博客，完成Hadoop的安装。
（3）安装Spark：需要在Linux系统上安装Spark，可以参考这篇博客，完成Hadoop的安装。
（4）安装Bottle：
Bottle 是一个非常小巧但高效的微型 Python Web 框架，它被设计为仅仅只有一个文件的Python模块，并且除Python标准库外，它不依赖于任何第三方模块。关于Bottle的更多介绍，可以参考这篇博客。
实验使用 web 框架 bottle 进行数据可视化，首先，打开一个Linux终端，运行下面命令安装 pip3 命令：

```
1. sudo apt-get install python3-pip
```

Shell 命令

然后，执行如下命令安装 bottle web 框架：

```
pip3 install bottle
```

Shell 命令

至此，完成环境搭建过程。

二、数据预处理

本次项目使用的数据集来自知名数据网站 Kaggle 的 tmdb-movie-metadata 电影数据集，该数据集包含大约 5000 部电影的相关数据。本次实验使用数据集中有关电影的数据表 tmdb_5000_movies.csv 进行实验。数据包含以下字段：

字段名称 解释 例子

budget 预算 10000000
genres 体裁 "[{""id"": 18, ""name"": ""Drama""}]"
homepage 主页 ""
id id 268238
keywords 关键词 "[{""id"": 14636, ""name"": ""india""}]"
original_language 原始语言 en
original_title 原标题 The Second Best Exotic Marigold Hotel
overview 概览 As the Best Exotic Marigold Hotel ...
popularity 流行度 17.592299
production_companies 生产公司 "[{""name"": ""Fox Searchlight Pictures"", ""id"": 43}, ...]"
production_countries 生产国家 "[{""iso31661"": ""GB"", ""name"": ""United Kingdom""}, ...]"
release_date 发行日期 2015-02-26
revenue 盈收 85978266
runtime 片长 122
spoken_languages 语言 "[{""iso6391"": ""en"", ""name"": ""English""}]"
status 状态 Released
tagline 宣传语 ""
title 标题 The Second Best Exotic Marigold Hotel
vote_average 平均分 6.3
vote_count 投票人数 272

由于数据中某些字段包含 json 数据，因此直接使用 DataFrame 进行读取会出现分割错误，所以如果要创建 DataFrame，需要先直接读

取文件生成 RDD，再将 RDD 转为 DataFrame。过程中，使用 python3 中的 csv 模块对数据进行解析和转换。

为了更方便的对 csv 文件转化的 RDD 进行处理，需要首先去除csv 文件的标题行。完成后，将处理好的文件 tmdb_5000_movies.csv 存储到 HDFS 上方便进一步的处理，使用下面命令将文件上传至 HDFS：

```
hdfs dfs -put tmdb_5000_movies.csv
```
Shell 命令

此时文件在 HDFS 上的路径为 /user/hadoop/tmdb_5000_movies.csv。之后在程序中，使用下面语句即可读取该文件：

```
sc.textFile('tmdb_5000_movies.csv')
```
tmdb_5000_movies.csv文件也可以直接从百度网盘下载，百度网盘地址：https://pan.baidu.com/s/1lt7PHF17-gHieOU0B0zJ3A 提取码：cui7

三、使用 Spark 将数据转为 DataFrame

由于读入的文件是 csv 文件，是结构化的数据，因此可以将数据创建为 DataFrame 方便进行分析。从本节开始，之后的代码均存放在 analyst.py 中，下面会逐行代码进行解释，解释完毕以后，再给出 analyst.py代码的全貌。（备注：所有代码文件可以直接从百度网盘下载，百度网盘地址：https://pan.baidu.com/s/1lt7PHF17-gHieOU0B0zJ3A 提取码：cui7 ）

为了创建 DataFrame，首先需要将 HDFS 上的数据加载成 RDD，再将 RDD 转化为 DataFrame。下面代码段完成从文件到 RDD 再到 DataFrame 的转化：

```
from pyspark import SparkContext
```

```
from pyspark.sql import SparkSession, Row
from pyspark.sql.types import StringType,
StructField, StructType
import json # 用于后面的流程
import csv

# 1. 创建 SparkSession 和 SparkContext 对象
sc = SparkContext('local', 'spark_project')
sc.setLogLevel('WARN') # 减少不必要的 LOG 输出
spark = SparkSession.builder.getOrCreate()

# 2. 为 RDD 转为 DataFrame 创建 schema
schemaString =
"budget,genres,homepage,id,keywords,original_langu
age,original_title,overview,popularity,production_
companies,production_countries,release_date,revenu
e,runtime,spoken_languages,status,tagline,title,vo
te_average,vote_count"
fields = [StructField(field, StringType(), True)
          for field in schemaString.split(")]
schema = StructType(fields)

# 3. 对于每一行用逗号分隔的数据，使用 csv 模块进行解析并转
为 Row 对象，得到可以转为 DataFrame 的 RDD
moviesRdd =
sc.textFile('tmdb_5000_movies.csv').map(
    lambda line: Row(*next(csv.reader([line]))))

# 4. 使用 createDataFrame 创建 DataFrame
mdf = spark.createDataFrame(moviesRdd, schema)
```

上述代码完成 4 件事：

首先，创建 SparkSession 和 SparkContext 对象。

然后，为 RDD 转为 DataFrame 制作表头 (schema)。schema 是一个 StructType 对象，该对象使用一个 StructField 数组创建。

每一个 StructField 都代表结构化数据中的一个字段，构造 StructField 需要 3 个参数

1. 字段名称
2. 字段类型
3. 字段是否可以为空
下面是这些对象的结构关系：

```
StructType([StructField(name, type, null), ..., StructField(name, type, null)])
```

接着，开始创建用于转为 DataFrame 的 RDD。这个过程首先读取 HDFS 上的数据文件，然后为了将 RDD 转为 DataFrame，还需要将数据的每一行转为一个 Row 对象。
这个过程首先使用 csv 模块进行解析，得到一个包含每个字段的迭代器：

```
csv.reader([line]) # 这里 [line] 表示包含一行数据的数组
```

然后使用 next 函数将迭代器中的数据读取到数组中：

```
next(csv.reader([line]))
```

最后使用 * 将数组转为 Row 对象的构造函数参数，创建 Row 对象：

```
Row(*next(csv.reader([line])))
```

至此，moviesRdd 中每一行为一个 Row 对象。
最后，通过 SparkSession 接口 createDataFrame ，使用准备好的表头 (schema) 和 RDD 创建 DataFrame：

```
mdf = spark.createDataFrame(moviesRdd, schema)
```

至此完成 DataFrame 的创建。

四、使用 Spark 进行数据分析

（备注：所有代码文件可以直接从百度网盘下载，百度网盘地址：https://pan.baidu.com/s/1lt7PHF17-gHieOU0B0zJ3A 提取码：cui7 ）
下面使用通过 Spark 处理得到的 DataFrame mdf 进行数据分析，首

先对数据中的主要字段单独进行分析（概览小节），然后再分析不同字段间的关系（关系小节）。
为了方便进行数据可视化，每个不同的分析，都将分析结果导出为 json 文件由 web 页面读取并进行可视化。导出直接使用下面的 save 函数：

```
def save(path, data):
  with open(path, 'w') as f:
    f.write(data)
```

该函数向 path 中写入 data。
下面分别介绍各个分析的生成过程。

1.概览

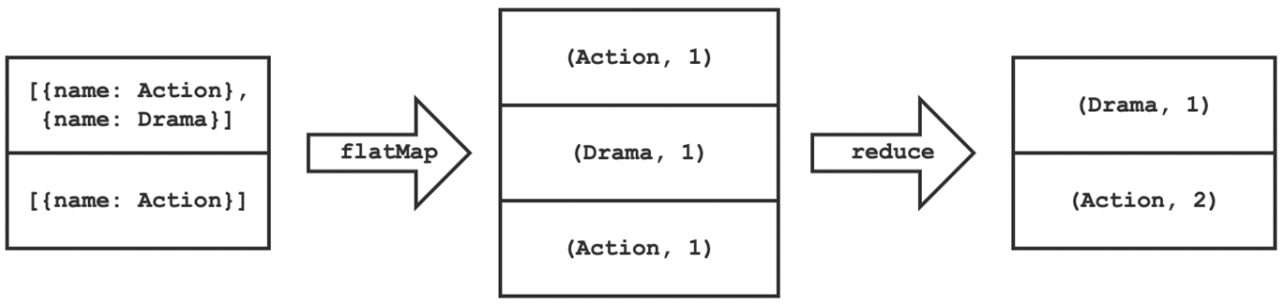
这个部分对数据进行整体的分析。

（1）TMDb 电影中的体裁分布

从上面的数据字典描述可以看出，电影的体裁字段是一个 json 格式的数据，因此，为了统计不同体裁的电影的数量，需要首先解析 json 数据，从中取出每个电影对应的体裁数组，然后使用词频统计的方法统计不同体裁出现的频率，即可得到电影的体裁分布。
首先实现一个函数 countByJson(field) ，该函数实现解析 json 格式字段从中提取出 name 并进行词频统计的功能：

```
def countByJson(field):
    return mdf.select(field).filter(mdf[field] != 
'').rdd.flatMap(lambda g: [(v, 1) for v in 
map(lambda x: x['name'], 
json.loads(g[field]))]).repartition(1).reduceByKey
(lambda x, y: x + y)
```

该函数返回一个 RDD，整个过程如下所示。



基于这个函数实现 countByGenres 用来生成不同体裁的电影数统计结果：

```
def countByGenres():
    res = countByJson("genres").collect()
    return list(map(lambda v: {"genre": v[0], "count": v[1]}, res))
```

这个函数调用 countByJson 得到频率统计结果，并将其转为 json 数据格式并返回，方便进行可视化。最终函数返回数据格式如下：

```
[{
    "genre": ...,
    "count": ...
}, {
    "genre": ...,
    "count": ...
}, ...]
```

接着，使用下面代码进行数据导出至 genres.json 方便之后进行可视化

```
save('genres.json', json.dumps(countByGenres())) # 确保 json 包已导入
```

2. 前 100 个常见关键词

该项分析电影关键词中出现频率最高的前一百个。由于关键词字段也是 json 格式数据，因此调用 countByJson 进行频率统计，同时对于统计结果进行降序排序并取前 100 项即可：

```
def countByKeywords():
    res = countByJson("keywords").sortBy(lambda x: -x[1]).take(100)
```

```
    return list(map(lambda v: {"x": v[0], "value": v[1]}, res))
```
最终该函数返回 json 数据格式如下：

```
[{
    "x": ...,
    "value": ...
}, {
    "x": ...,
    "value": ...
}, ...]
```
接着，使用下面代码将数据导出至 keywords.json 方便之后进行可视化

```
save('keywords.json', json.dumps(countByKeywords()))
```
3. TMDb 中最常见的 10 种预算数

这一项探究电影常见的预算数是多少，因此需要对电影预算进行频率统计，代码如下：

```
def countByBudget(order='count', ascending=False):
    return mdf.filter(mdf["budget"] != 0).groupBy("budget").count().orderBy(order, ascending=ascending).toJSON().map(lambda j: json.loads(j)).take(10)
```
首先，需要对预算字段进行过滤，去除预算为 0 的项目，然后根据预算聚合并计数，接着根据计数进行排序，并将结果导出为 json 字符串，为了统一输出，这里将 json 字符串转为 python 对象，最后取前 10 项作为最终的结果。
最终该函数返回 json 数据格式如下：

```
[{
    "budget": ...,
    "count": ...
}, {
    "budget": ...,
```

```
    "count": ...
}, ...]
```
接着，使用下面代码进行数据导出至 budget.json 方便之后进行可视化

```
save('budget.json', json.dumps(countByBudget()))
```
4. TMDb 中最常见电影时长 (只展示电影数大于 100 的时长)

这一项统计 TMDb 中最常见的电影时长，首先，需要过滤时长为 0 的电影，然后根据时长字段聚合并计数，接着过滤掉出现频率小于 100 的时长 （这一步是为了方便可视化，避免过多冗余信息）得到最终的结果。

```
def distrbutionOfRuntime(order='count', ascending=False):
    return mdf.filter(mdf["runtime"] != 0).groupBy("runtime").count().filter('count>=100').toJSON().map(lambda j: json.loads(j)).collect()
```
最终该函数返回 json 数据格式如下：

```
[{
    "runtime": ...,
    "count": ...
}, {
    "runtime": ...,
    "count": ...
}, ...]
```
接着，使用下面代码进行数据导出至 runtime.json 方便之后进行可视化

```
save('runtime.json', json.dumps(distrbutionOfRuntime()))
```
5. 生产电影最多的 10 大公司

这一项统计电影产出最多的 10 个公司，同样使用 countByJson 对 JSON 数据进行频率统计，然后进行降序排列取前 10 项即可。

```
def countByCompanies():
    res = countByJson("production_companies").sortBy(lambda x: -x[1]).take(10)
    return list(map(lambda v: {"company": v[0], "count": v[1]}, res))
```

最终该函数返回 JSON 数据格式如下：

```
[{
    "company": ...,
    "count": ...
}, {
    "company": ...,
    "count": ...
}, ...]
```

接着，使用下面代码进行数据导出至 company_count.json 方便之后进行可视化

```
save('company_count.json', json.dumps(countByCompanies()))
```

6. TMDb 中的 10 大电影语言

该项统计 TMDb 中出现最多的语言，与前面类似，该字段也是 JSON 数据，因此首先对每个项目进行词频统计，然后过滤掉语言为空的项目，最后排序取前十即可。

```
def countByLanguage():
    res = countByJson("spoken_languages").filter(
        lambda v: v[0] != '').sortBy(lambda x: -x[1]).take(10)
    return list(map(lambda v: {"language": v[0], "count": v[1]}, res))
```

最终该函数返回 json 数据格式如下：

```
[{
```

```
    "language": ...,
    "count": ...
}, {
    "language": ...,
    "count": ...
}, ...]
```
接着，使用下面代码进行数据导出至 language.json 方便之后进行可视化

```
save('language.json',
json.dumps(countByLanguage()))
```
2.关系

这个部分考虑数据之间的关系。

1. 预算与评价的关系

这部分考虑预算与评价之间的关系，因此对于每个电影，需要导出如下的数据：

```
[电影标题，预算，评价]
```
基于 DataFrame 对数据进行字段过滤即可：

```
def budgetVote():
    return mdf.select("title", "budget",
"vote_average").filter(mdf["budget"] !=
0).filter(mdf["vote_count"] > 100).collect()
```
这里还要注意过滤掉预算为空的数据，同时，结果只保留了投票数大于 100 的数据确保公平。
得到的数据存储在 budget_vote.json 中：

```
save('budget_vote.json', json.dumps(budgetVote()))
```
2. 发行时间与评价的关系

这部分考虑发行时间与评价之间的关系，因此对于每个电影，需要导出如下的数据：

[电影标题，发行时间，评价]
基于 DataFrame 对数据进行字段过滤即可：

```
def dateVote():
    return mdf.select(mdf["release_date"], "vote_average", "title").filter(mdf["release_date"] != "").filter(mdf["vote_count"] > 100).collect()
```

这里还是要注意过滤掉发行时间为空的数据，保留投票数大于 100 的数据。
得到的数据存储在 date_vote.json 中：

```
save('date_vote.json', json.dumps(dateVote()))
```

3. 流行度和评价的关系

这部分考虑流行度与评价之间的关系，因此对于每个电影，需要导出如下的数据：

[电影标题，流行度，评价]
基于 DataFrame 对数据进行字段过滤即可：

```
def popVote():
    return mdf.select("title", "popularity", "vote_average").filter(mdf["popularity"] != 0).filter(mdf["vote_count"] > 100).collect()
```

同时，过滤掉流行度为 0 的数据，保留投票数大于 100 的数据。
得到的数据存储在 pop_vote.json 中：

```
save('pop_vote.json', json.dumps(popVote()))
```

4. 公司生产的电影平均分和数量的关系

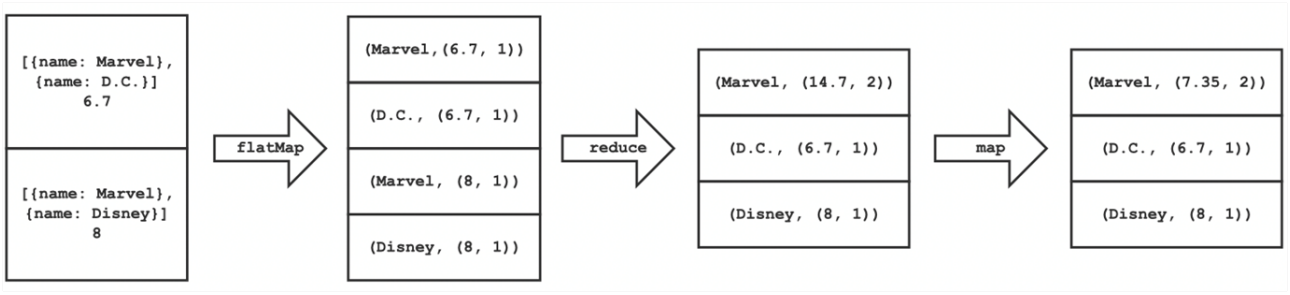
这部分计算每个公司生产的电影数量及这些电影的平均分分布。首先，需要对数据进行过滤，去掉生产公司字段为空和评价人数小于 100 的电影，然后对于每一条记录，得到一条如下形式的记录：

[公司名，(评分，1)]

接着将所有记录的评分和计数累加，最后用总评分除以计数得到一个公司的平均评分及电影数，整个过程如下所示。

```
def moviesVote():
    return
mdf.filter(mdf["production_companies"] !=
'').filter(mdf["vote_count"] >
100).rdd.flatMap(lambda g: [(v,
[float(g['vote_average']), 1]) for v in map(lambda
x: x['name'],
json.loads(g["production_companies"]))]).repartiti
on(1).reduceByKey(lambda x, y: [x[0] + y[0], x[1]
+ y[1]]).map(lambda v: [v[0], v[1][0] / v[1][1],
v[1][1]]).collect()
```

下面是过程的图示



得到的数据存储在 movies_vote.json 中：

```
save('movies_vote.json', json.dumps(moviesVote()))
```

5. 电影预算和营收的关系

这部分考虑电影的营收情况，因此对于每个电影，需要导出如下的数据：

```
[电影标题，预算，收入]
```

基于 DataFrame 对数据进行字段过滤即可：

```
def budgetRevenue():
    return mdf.select("title", "budget",
"revenue").filter(mdf["budget"] !=
0).filter(mdf['revenue'] != 0).collect()
```

过滤掉预算，收入为 0 的数据。
得到的数据存储在 budget_revenue.json 中：

```
save('budget_revenue.json',
json.dumps(budgetRevenue()))
```

3.小结

最后，将上面的过程整合起来方便进行调用，因此在 analyst.py 中添加 main 函数：

```
if __name__ == "__main__":
    m = {
        "countByGenres": {
            "method": countByGenres,
            "path": "genres.json"
        },
        "countByKeywords": {
            "method": countByKeywords,
            "path": "keywords.json"
        },
        "countByCompanies": {
            "method": countByCompanies,
            "path": "company_count.json"
        },
        "countByBudget": {
            "method": countByBudget,
            "path": "budget.json"
        },
        "countByLanguage": {
            "method": countByLanguage,
            "path": "language.json"
        },
        "distrbutionOfRuntime": {
            "method": distrbutionOfRuntime,
            "path": "runtime.json"
        },
        "budgetVote": {
```

```
            "method": budgetVote,
            "path": "budget_vote.json"
        },
        "dateVote": {
            "method": dateVote,
            "path": "date_vote.json"
        },
        "popVote": {
            "method": popVote,
            "path": "pop_vote.json"
        },
        "moviesVote": {
            "method": moviesVote,
            "path": "movies_vote.json"
        },
        "budgetRevenue": {
            "method": budgetRevenue,
            "path": "budget_revenue.json"
        }
    }
    base = "static/" # 生成文件的 base 目录
    if not os.path.exists(base): # 如果目录不存在则创建一个新的
        os.mkdir(base)

    for k in m: # 执行上述所有方法
        p = m[k]
        f = p["method"]
        save(base + m[k]["path"], json.dumps(f()))
        print ("done -> " + k + " , save to -> " + base + m[k]["path"])
```

上面代码将所有的函数整合在变量 m中，然后通过循环调用上述所有方法并导出json文件。

通过下面方法进行数据分析：

```
spark-submit analyst.py
```

Shell 命令

五、数据可视化方法

（备注：所有代码文件可以直接从百度网盘下载，百度网盘地址：https://pan.baidu.com/s/1lt7PHF17-gHieOU0B0zJ3A 提取码：cui7 ）

数据可视化基于阿里开源的数据可视化工具 G2 实现。G2 是一套基于可视化编码的图形语法，以数据驱动，具有高度的易用性和扩展性，用户无需关注各种繁琐的实现细节，一条语句即可构建出各种各样的可交互的统计图表。下面以 TMDb 中电影的体裁分布为例说明可视化过程。

首先使用 python Web 框架 bottle 访问可视化页面方便进行 json 数据的读取。使用下面代码web.py 可以实现一个简单的静态文件读取：

```
from bottle import route, run, static_file
import json

@route('/static/<filename>')
def server_static(filename):
  return static_file(filename, root="./static")

@route("/<name:re:.*\.html>")
def server_page(name):
  return static_file(name, root=".")

@route("/")
def index():
  return static_file("index.html", root=".")


run(host="0.0.0.0", port=9999)
```

bottle 对于接收到的请求进行路由

1. 对于 web 服务启动目录中 static 文件夹下的文件，直接返回对应文件名的文件；
2. 对于启动目录下的 html 文件，也返回对应的页面。

3. 直接访问本机的 9999 端口，则返回主页。
最后，将 web 服务绑定到本机的 9999 端口。根据上面的实现，对于 web 页面 (html 文件)，直接放在服务启动的目录下，对于 Spark 分析的结果，则保存在 static 目录下。
接下来实现主页文件 index.html。

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-
width,height=device-height">
  <title>TMDb 电影数据分析</title>
  <style>
    /* 这里省略 */
  </style>
</head>

<body>
  <div class="container">
    <h1 style="font-size: 40px;"># TMDb Movie Data
Analysis <br> <small style="font-size: 55%;color:
rgba(0,0,0,0.65);">>
        Big Data Processing Technology on Spark</
small> </h1>
    <hr>
    <h1 style="font-size: 30px;color: #404040;">I.
Overviews</h1>
    <div class="chart-group">
      <h2>- Distribution of Genres in TMDb <br>
<small style="font-size: 72%;">> This figure
          compares the genre
          distribution in TMDb, and you can see
that most of the movies in TMDb is Drama.</small>
</h2>
```

```
      <iframe src="genres.html" class="frame"
frameborder="0"></iframe>
    </div>
  </div>

  <script>/*Fixing iframe window.innerHeight 0
issue in Safari*/document.body.clientHeight;</
script>
</body>

</html>
```
每个图表通过一个 iframe 引入到主页中。对于每一个图表，主页中都包含标题和图表所在的页面的 iframe。对于 TMDb 中的体裁分布分析结果，在 genres.html 中实现，下面对该文件进行实现。

```
<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-
width,height=device-height">
  <title>TOP 5000 电影数据分析</title>
  <style>
    ::-webkit-scrollbar {
      display: none;
    }

    html,
    body {
      font-family: 'Ubuntu Condensed';
      height: 100%;
      margin: 0;
      color: rgba(0, 0, 0, 0.85);
    }
  </style>
</head>
```

```
<body>
  <div id="mountNode"></div>
  </div>
  <script>/*Fixing iframe window.innerHeight 0 issue in Safari*/document.body.clientHeight;</script>
  <script src="static/g2.min.js"></script>
  <script src="static/data-set.min.js"></script>
  <script src="static/jquery-3.2.1.min.js"></script>
  <script>
    function generateChart(id, type, xkey, xlabel, ykey, ylabel) {
      var chart = new G2.Chart({ // 初始化 chart
        container: id,
        forceFit: true,
        height: 500,
        padding: [40, 80, 80, 80],
      });
      chart.scale(ykey, { // 对 y 尺度进行设置
        alias: ylabel,
        min: 0,
        // max: 3000,
        tickCount: 4
      });

      chart.axis(xkey, { // 对 x 坐标轴设置
        label: {
          textStyle: {
            fill: '#aaaaaa'
          }
        },
        tickLine: {
          alignWithLabel: false,
          length: 0
        }
```

```
      });

      chart.axis(ykey, { // 对 y 坐标轴设置
        label: {
          textStyle: {
            fill: '#aaaaaa'
          }
        },
        title: {
          offset: 50
        }
      });
      chart.legend({    // 设置图例
        position: 'top-center'
      });
        //设置标签和颜色等
      chart.interval().position(`${xkey}*${ykey}
`).label(ykey).color('#ffb877').opacity(1).adjust(
[{
        type,
        marginRatio: 1 / 32
      }]);
      chart.render();

      return chart;
    }
  </script>
  <script>
    // 调用上述函数创建图表
    let chart = generateChart('mountNode',
'dodge', 'genre', 'genres', 'count', '# movies');

    window.onload = () => {
      // 当页面加载后使用 jQuery 提供的方法进行json文件
的读取。
      $.getJSON("/static/genres.json", d => {
```

```
        chart.changeData(d) // 使用 chart 的更新数据 API 进行数据更新。
      })
    }
  </script>
</body>

</html>
```

代码的过程解释以注释给出，使用该页面前，还需要将对应的 js 库(g2.js, data-set.js, jquery)放入到 static 文件夹下。
代码完成后，在代码所在的根目录下，执行：

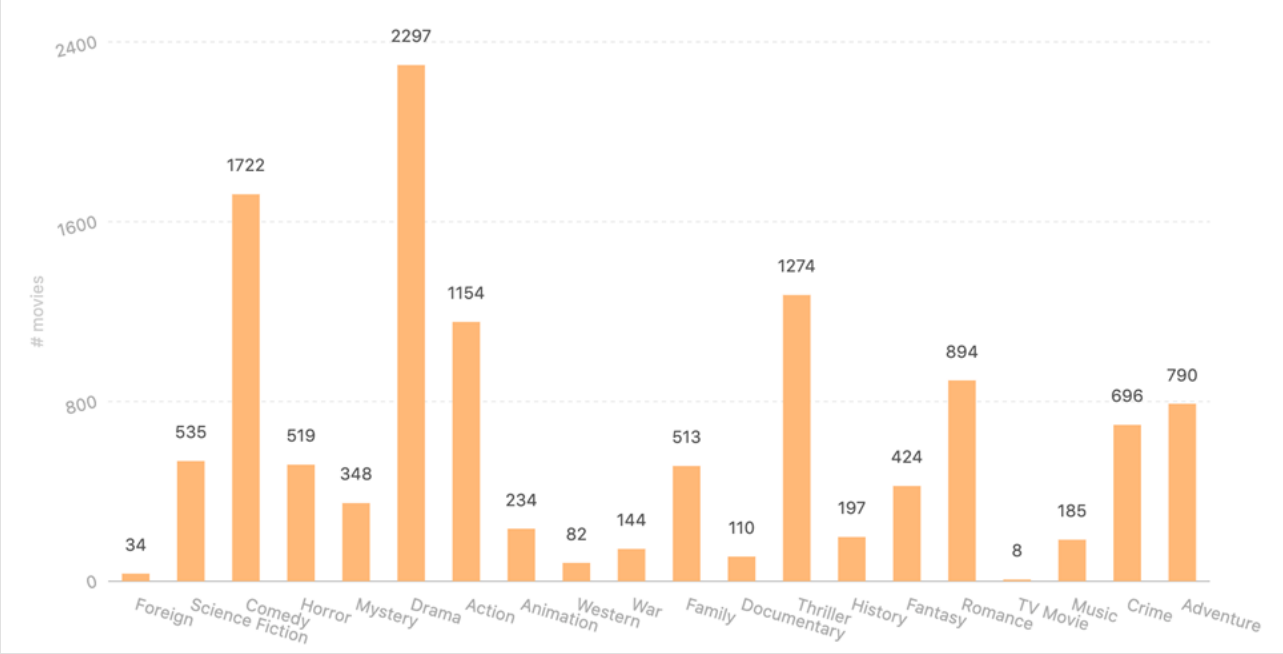
```
spark-submit web.py # 或者执行 python3.5 web.py
```

Shell 命令

命令行出现：

```
Bottle v0.12.16 server starting up (using WSGIRefServer())...
Listening on http://0.0.0.0:9999/
Hit Ctrl-C to quit.
```

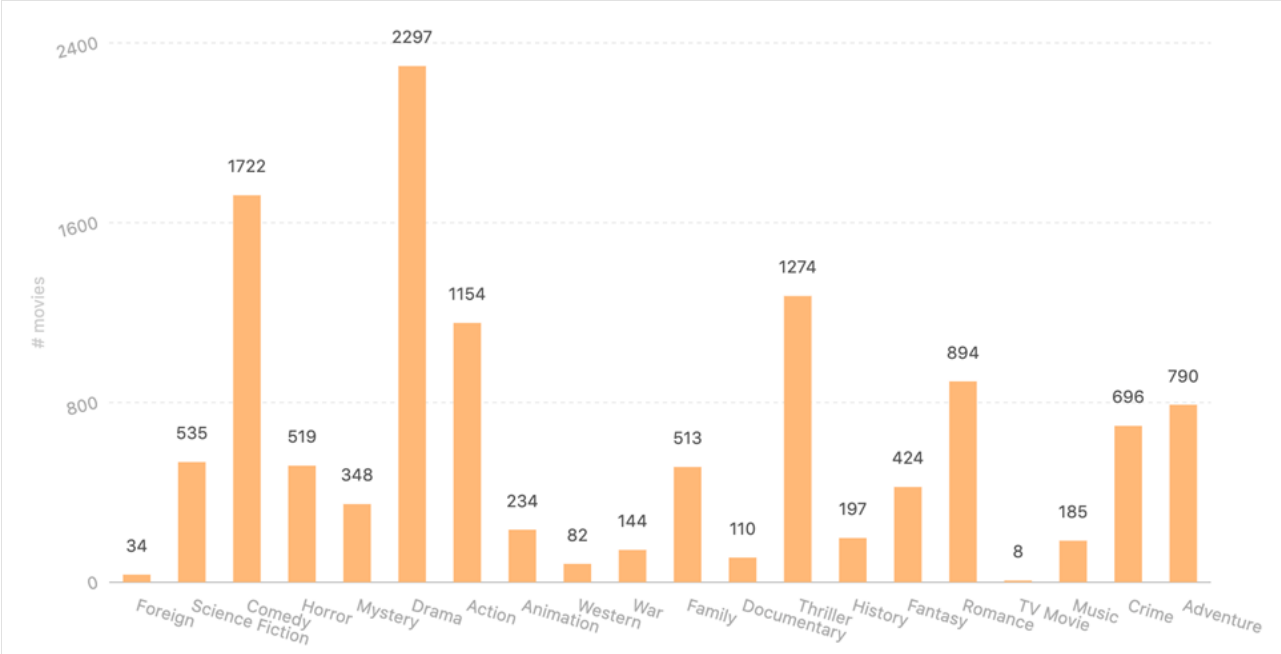
即完成启动，打开浏览器访问 http://127.0.0.1:9999 即可看到可视化结果，下面是图表的例子：



六、数据图表

（一）概览

1. TMDb 电影中的体裁分布

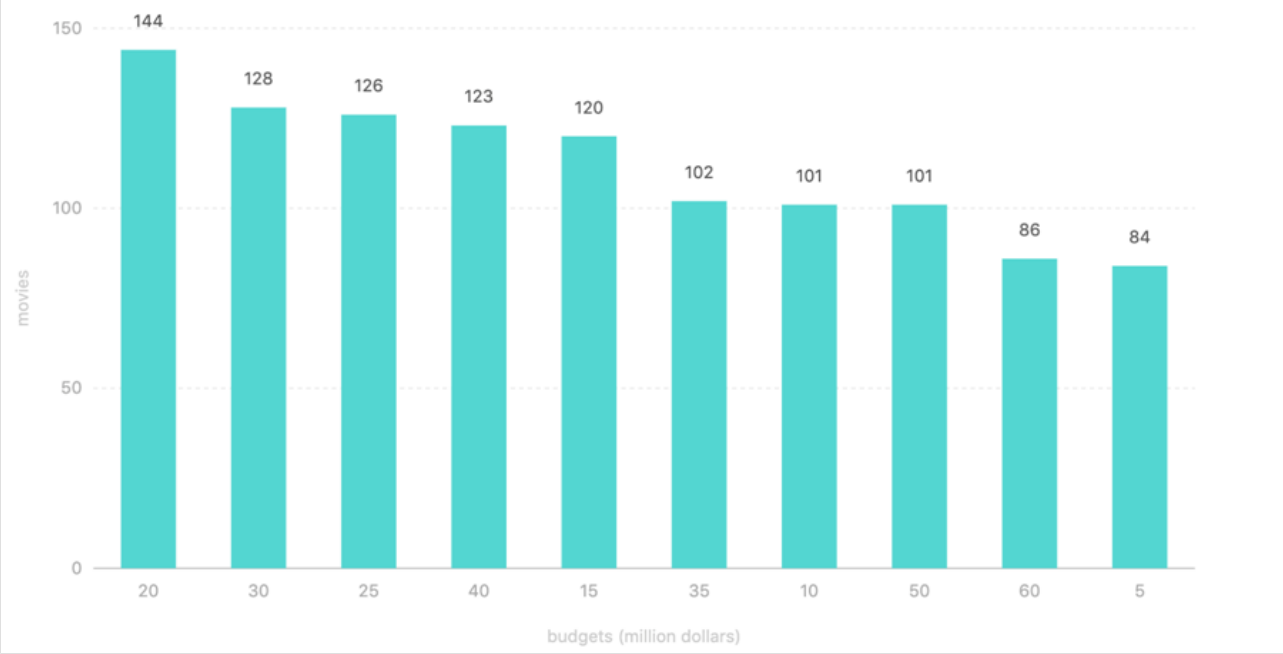


2. 从图中可以看出，Drama 的电影在 TMDb 中占比较大，其次 Science Fiction、Action 和 Thriller 的数量也较多。
3. 前 100 个常见关键词



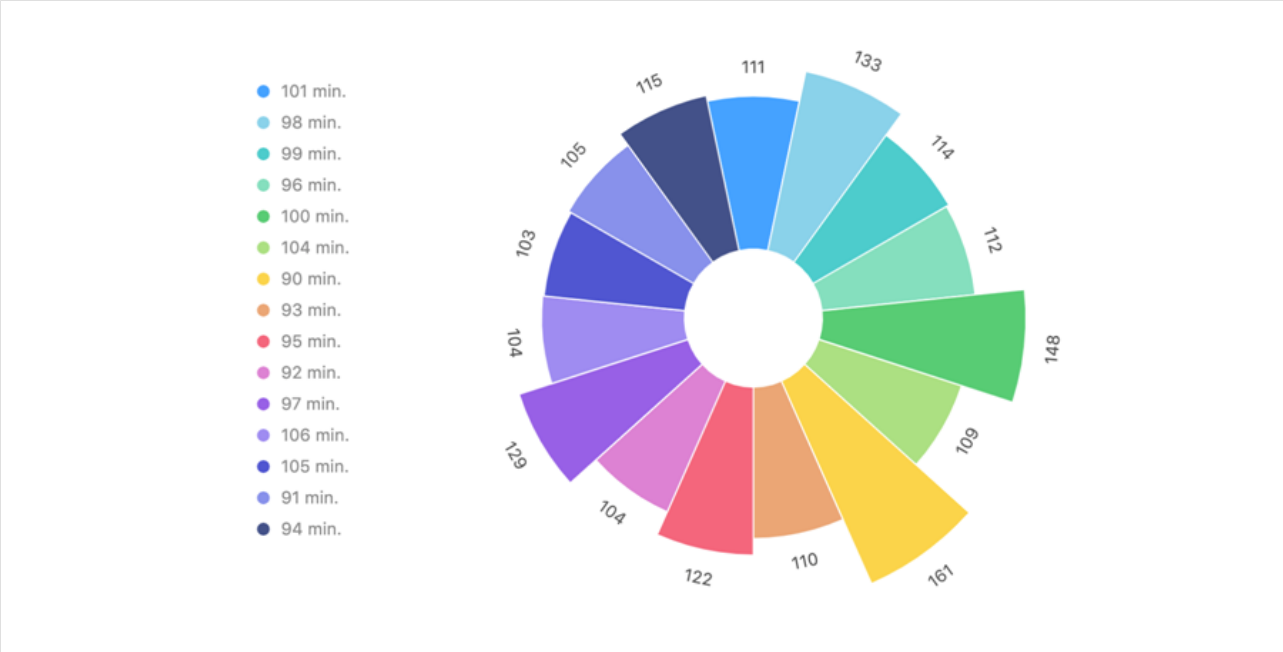
TMDb 中最常见的关键词是 Woman Director，其次还有 independent film 等。

3. TMDb 中最常见的 10 种预算数



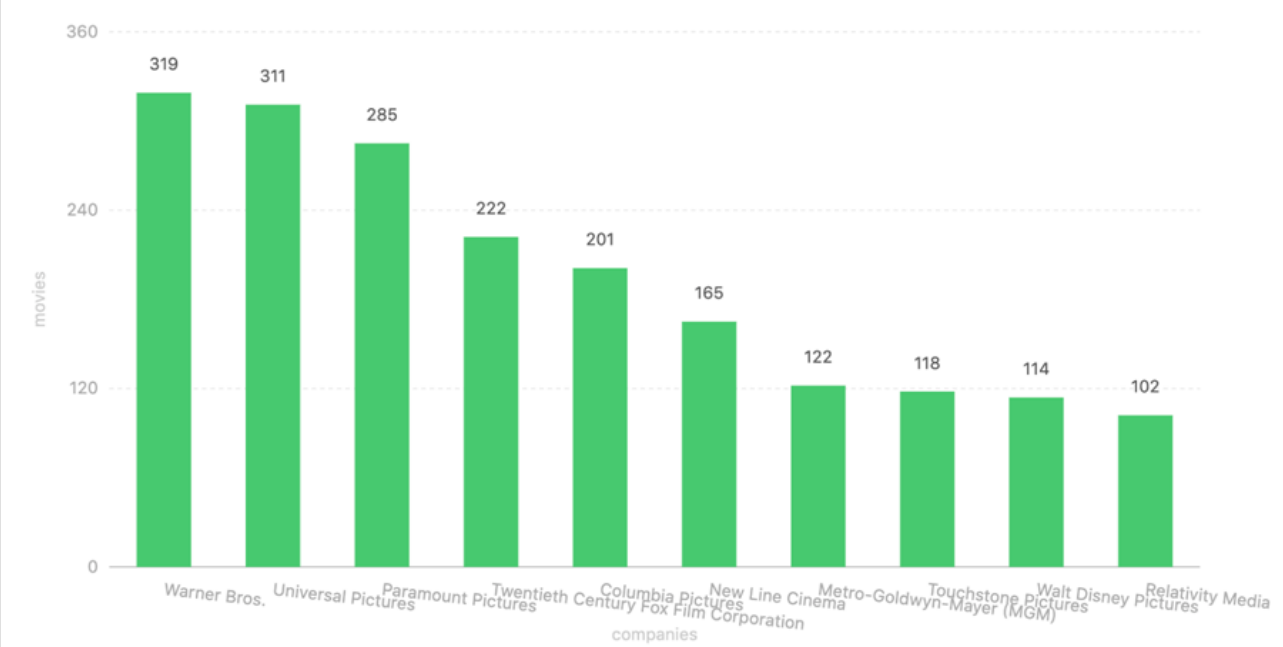
有 144 部电影的预算为 20,000,000，是最常见的预算值。

4. TMDb 中最常见电影时长 (只展示电影数大于 100 的时长)



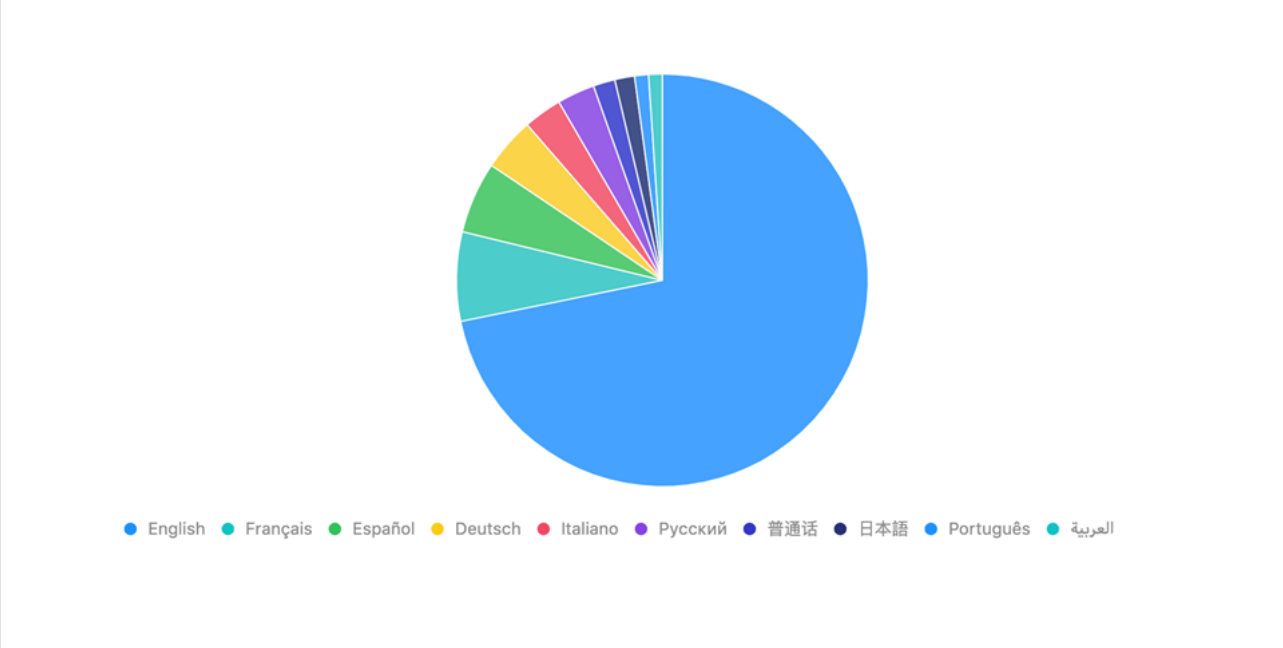
多数电影的时长是90分钟或100分钟。

5. 生产电影最多的 10 大公司



生产电影较多的公司是 Warner Bros.、Universal Pictures等。

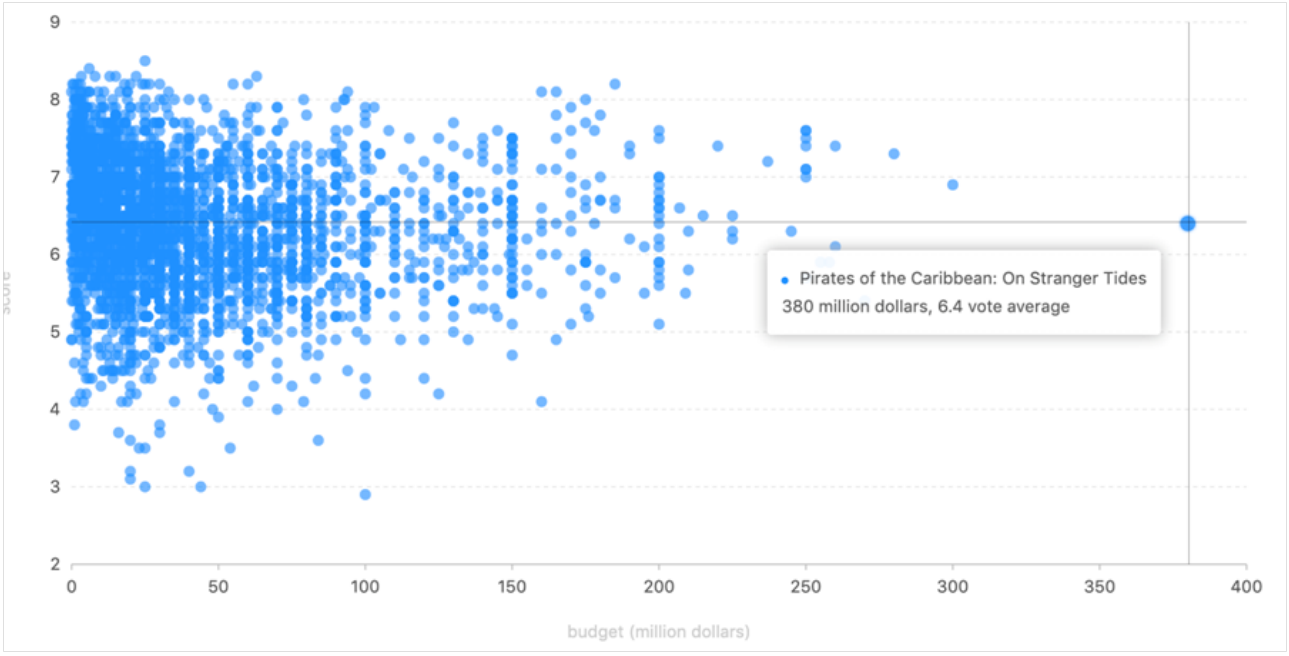
6. TMDb 中的 10 大电影语言



大多数电影中的语言是英语。

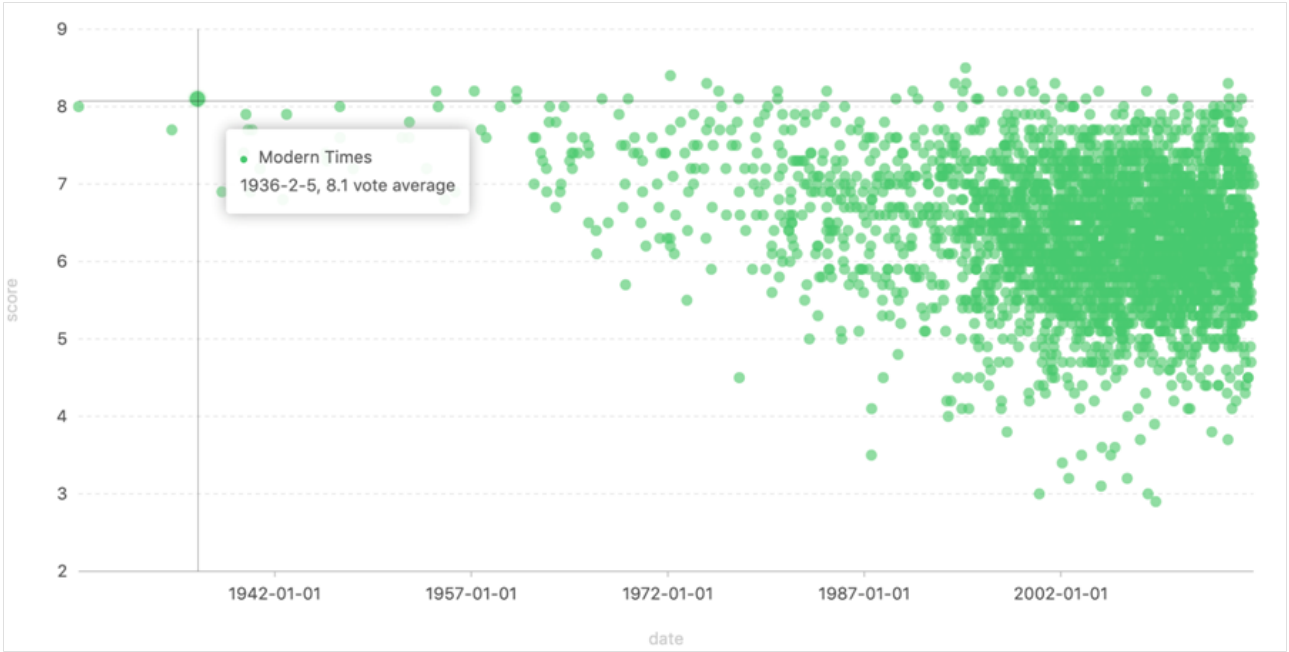
（二）关系

1. 预算与评价的关系



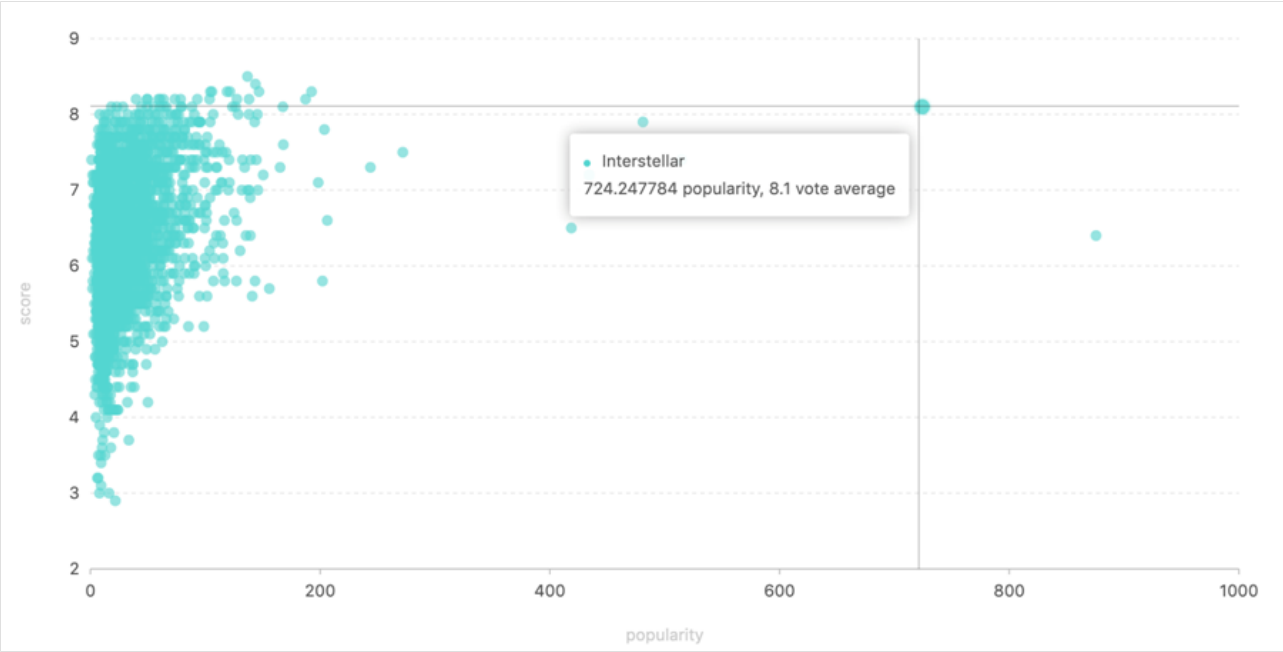
预算高的电影不见得能取得更好的评价，例如预算高达 380,000,000 美元的 Pirates of the Caribbean: On Stranger Tides（加勒比海盗）评价只有6.4分。

1. 发行时间与评价的关系



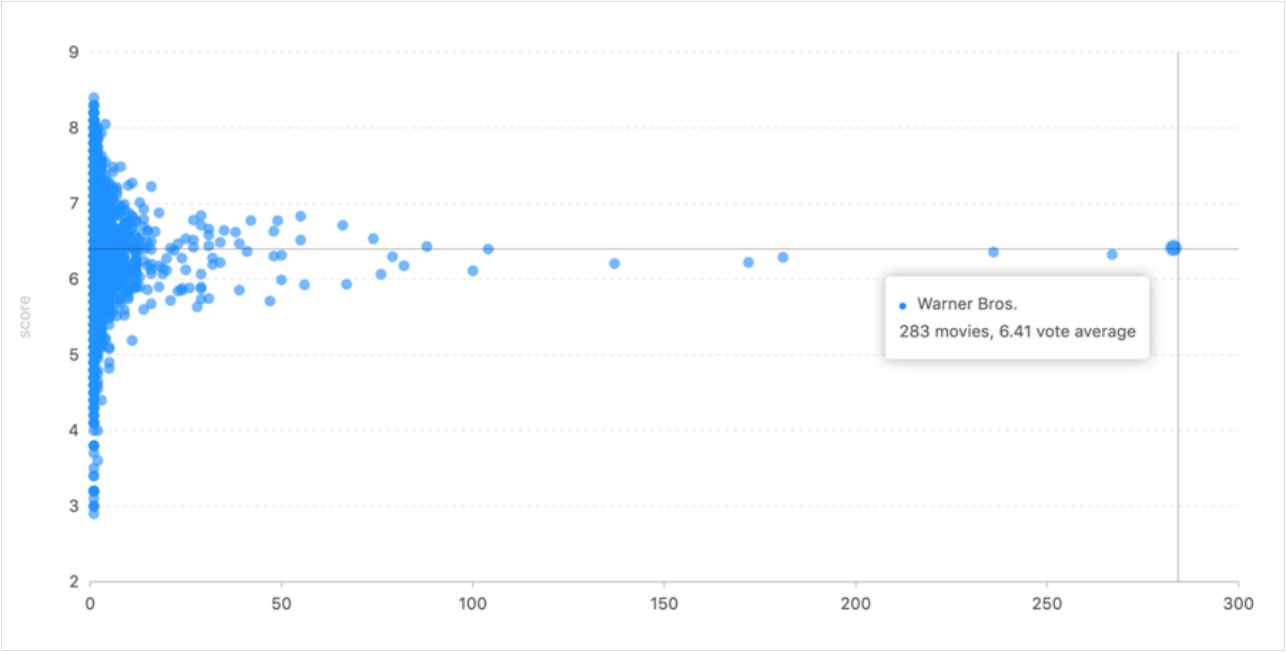
早期的电影评价都比较高，例如发行于1936年的 Modern Times（摩登时代）评价高达8.1分。

1. 流行度和评价的关系

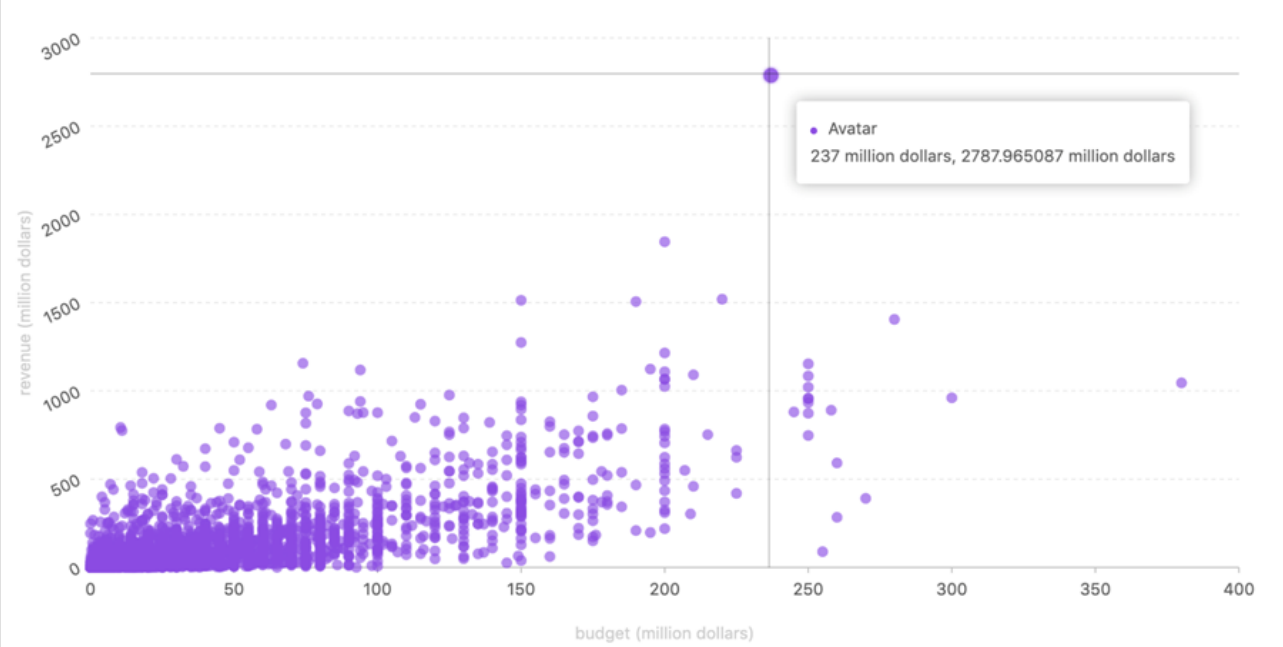


流行度较高的话一般能取得平均水平以上的评价，例如 Interstellar（星际穿越）流行度很高，评价为8.1分。

1. 公司生产的电影平均分和数量的关系



2. 从图中可以看出，一个公司生产的电影越多，其电影平均分越接近整体的平均水平。
3. 电影预算和营收的关系



从图中可以看出，多数电影都能实现正收入，而预算为 237,000,000 美元的 Avatar（阿凡达）最终收入为2787,965,087美元